HDFC Bank DCF Valuation Model Project

New section

```
!pip -q install yfinance plotly openpyxl
```

Imports

```
import warnings
warnings.filterwarnings('ignore')
import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import plotly.express as px
plt.style.use('ggplot')
```

```
def in_crore(x):
    return x / 1e7      # 1 Crore = 10,000,000
```

Configuration

```
TICKER='HDFCBANK.NS'
PERIOD='5y'
EXPORT=True
```

Download Data

```
ticker=yf.Ticker(TICKER)
price=ticker.history(period=PERIOD)
info=ticker.info
income=ticker.financials.T
balance=ticker.balance_sheet.T
cashflow=ticker.cashflow.T
display(price.tail())
```

Date	Open	High	Low	Close	Volume	Dividends	Stock Splits
2026-07-23 00:00:00+05:30	749.400024	751.000000	743.250000	747.250000	32532090	0.0	0.0
2026-07-24 00:00:00+05:30	738.500000	748.250000	737.250000	742.799988	23837859	0.0	0.0
2026-07-27 00:00:00+05:30	746.099976	747.950012	738.099976	739.549988	29678752	0.0	0.0
2026-07-28 00:00:00+05:30	732.000000	740.250000	732.000000	735.400024	33886037	0.0	0.0
2026-07-29 00:00:00+05:30	743.000000	751.200012	740.799988	750.400024	11139161	0.0	0.0

Cleaning

```
def clean(df):
    return df.sort_index().fillna(0)
income=clean(income)
balance=clean(balance)
cashflow=clean(cashflow)
```

Export

```
if EXPORT:
    income.to_csv('IncomeStatement.csv')
    balance.to_csv('BalanceSheet.csv')
    cashflow.to_csv('CashFlow.csv')
    price.to_csv('HistoricalPrices.csv')
print('Done')
```

```
Done
```

```
!pip -q install yfinance
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import yfinance as yf
plt.style.use('ggplot')
```

Load Financial Statements

```
ticker=yf.Ticker('HDFCBANK.NS')
income=ticker.financials.T.fillna(0)
balance=ticker.balance_sheet.T.fillna(0)
cashflow=ticker.cashflow.T.fillna(0)

# Display in Rs. Crore for readability
income_display = income.apply(lambda col: col / 1e7)
display(income_display.head().style.format("₹{:,.2f} Cr"))
```

Helper Functions

```
def forecast_growth(last_value,growth,n=5):
    vals=[]
    x=last_value
    for _ in range(n):
        x*=1+growth
        vals.append(x)
    return vals
```

Revenue Forecast

```
try:
    rev=income['Total Revenue'].sort_index()
except:
    rev=income.iloc[:,0]
```

```
hist=rev.values
g=pd.Series(hist).pct_change().mean()
future=forecast_growth(hist[-1],g,5)
forecast=pd.DataFrame({'Forecast Revenue':future},
index=[f'Year {i}' for i in range(1,6)])
forecast_display = forecast.copy()
forecast_display["Forecast Revenue"] = forecast_display["Forecast Revenue"].apply(in_crore)

forecast_display = forecast_display.style.format({
    "Forecast Revenue": "₹{:,.2f} Cr"
})

display(forecast_display)
```

	Forecast Revenue
Year 1	₹318,127.26 Cr
Year 2	₹426,753.56 Cr
Year 3	₹572,470.89 Cr
Year 4	₹767,944.20 Cr
Year 5	₹1,030,162.95 Cr

Operating Margin Assumption

```
margin=0.32
forecast['Operating Income']=forecast['Forecast Revenue']*margin

forecast_display = forecast.copy() / 1e7
display(forecast_display.style.format("₹{:,.2f} Cr"))
```

	Forecast Revenue	Operating Income
Year 1	₹318,127.26 Cr	₹101,800.72 Cr
Year 2	₹426,753.56 Cr	₹136,561.14 Cr
Year 3	₹572,470.89 Cr	₹183,190.68 Cr
Year 4	₹767,944.20 Cr	₹245,742.15 Cr
Year 5	₹1,030,162.95 Cr	₹329,652.14 Cr

CapEx, Depreciation & Tax

```
forecast['CapEx']=forecast['Forecast Revenue']*0.04
forecast['Depreciation']=forecast['Forecast Revenue']*0.015
forecast['Tax']=forecast['Operating Income']*0.25

forecast_display = forecast.copy() / 1e7
display(forecast_display.style.format("₹{:,.2f} Cr"))
```

	Forecast Revenue	Operating Income	CapEx	Depreciation	Tax
Year 1	₹318,127.26 Cr	₹101,800.72 Cr	₹12,725.09 Cr	₹4,771.91 Cr	₹25,450.18 Cr
Year 2	₹426,753.56 Cr	₹136,561.14 Cr	₹17,070.14 Cr	₹6,401.30 Cr	₹34,140.28 Cr
Year 3	₹572,470.89 Cr	₹183,190.68 Cr	₹22,898.84 Cr	₹8,587.06 Cr	₹45,797.67 Cr
Year 4	₹767,944.20 Cr	₹245,742.15 Cr	₹30,717.77 Cr	₹11,519.16 Cr	₹61,435.54 Cr
Year 5	₹1,030,162.95 Cr	₹329,652.14 Cr	₹41,206.52 Cr	₹15,452.44 Cr	₹82,413.04 Cr

Free Cash Flow

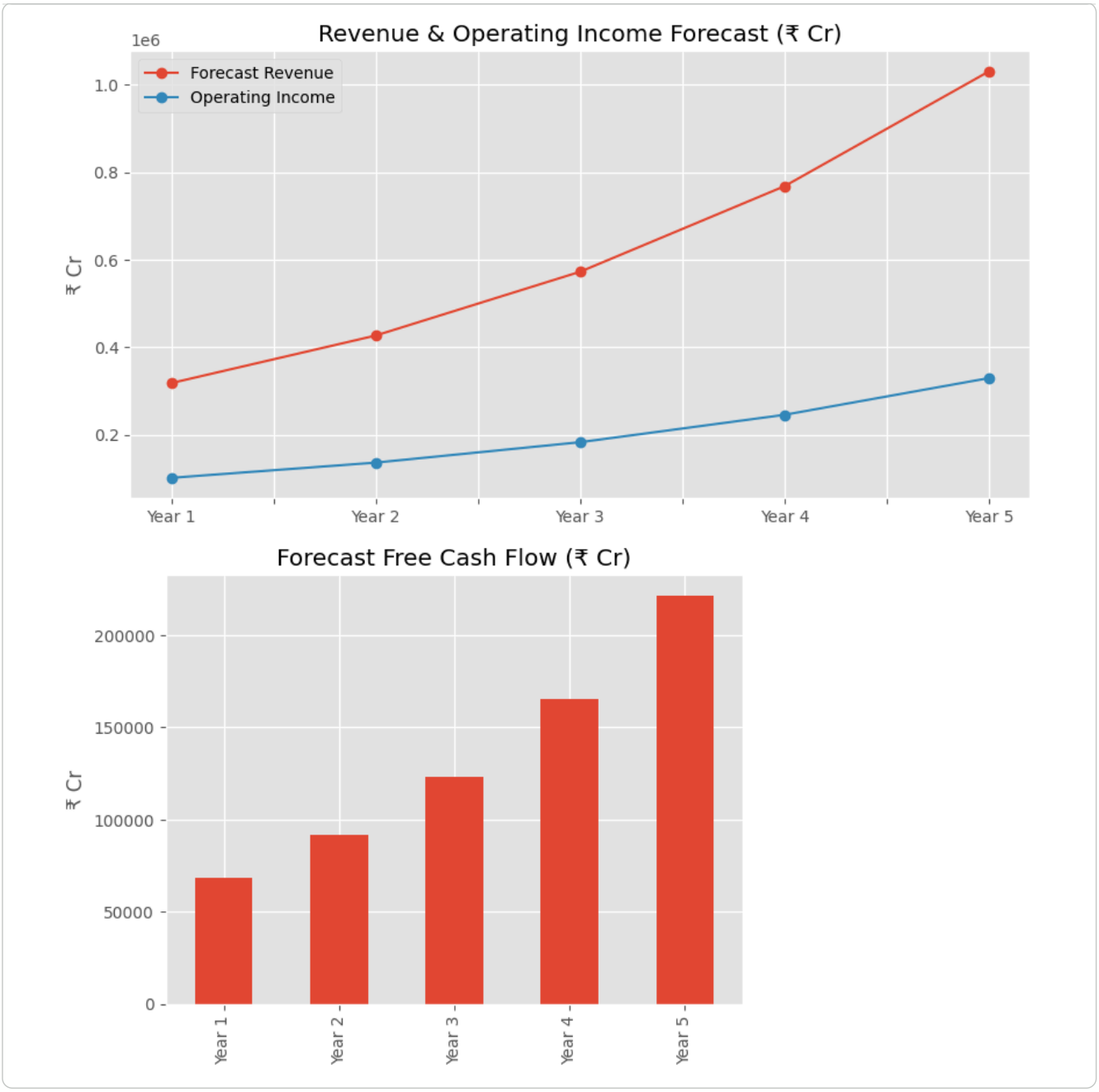
```
forecast['FCF']=forecast['Operating Income']-forecast['Tax']+forecast['Depreciation']-forecast['CapEx']
```

```
forecast_display = forecast.copy() / 1e7
display(forecast_display.style.format("₹{:,.2f} Cr"))
```

	Forecast Revenue	Operating Income	CapEx	Depreciation	Tax	FCF
Year 1	₹318,127.26 Cr	₹101,800.72 Cr	₹12,725.09 Cr	₹4,771.91 Cr	₹25,450.18 Cr	₹68,397.36 Cr
Year 2	₹426,753.56 Cr	₹136,561.14 Cr	₹17,070.14 Cr	₹6,401.30 Cr	₹34,140.28 Cr	₹91,752.01 Cr
Year 3	₹572,470.89 Cr	₹183,190.68 Cr	₹22,898.84 Cr	₹8,587.06 Cr	₹45,797.67 Cr	₹123,081.24 Cr
Year 4	₹767,944.20 Cr	₹245,742.15 Cr	₹30,717.77 Cr	₹11,519.16 Cr	₹61,435.54 Cr	₹165,108.00 Cr
Year 5	₹1,030,162.95 Cr	₹329,652.14 Cr	₹41,206.52 Cr	₹15,452.44 Cr	₹82,413.04 Cr	₹221,485.03 Cr

Visualisations

```
forecast_cr = forecast / 1e7

forecast_cr[['Forecast Revenue','Operating Income']].plot(figsize=(10,5),marker='o')
plt.title('Revenue & Operating Income Forecast (₹ Cr)')
plt.ylabel('₹ Cr')
plt.show()

forecast_cr['FCF'].plot(kind='bar',title='Forecast Free Cash Flow (₹ Cr)')
plt.ylabel('₹ Cr')
plt.show()
```



Export

```
forecast.to_csv('Forecast_Model.csv')
print('Forecast_Model.csv exported')
```

```
Forecast_Model.csv exported
```

```
!pip -q install yfinance scipy
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
```

Load Forecast

```
forecast=pd.read_csv('Forecast_Model.csv',index_col=0)
forecast_display = forecast.copy()

forecast_display['FCF'] = forecast_display['FCF'] / 1e7
```

```
display(forecast_display.style.format({
    'FCF': '₹{:,.2f} Cr'
}))
```

	Forecast Revenue	Operating Income	CapEx	Depreciation	Tax	
Year 1	3181272634865.780762	1018007243157.050049	127250905394.631256	47719089522.986717	254501810789.262512	₹68
Year 2	4267535561271.039062	1365611379606.732422	170701422450.841553	64013033419.065582	341402844901.683105	₹91
Year 3	5724708900179.278320	1831906848057.368896	228988356007.171173	85870633502.689178	457976712014.342285	₹123
Year 4	7679442038915.540039	2457421452452.972656	307177681556.621582	115191630583.733093	614355363113.243164	₹165
Year 5	10301629490228.998047	3296521436873.279785	412065179609.159912	154524442353.434967	824130359218.319824	₹221

Assumptions

```
risk_free=0.07
beta=1.0
market_return=0.13
cost_debt=0.08
tax_rate=0.25
debt_weight=0.20
equity_weight=0.80
terminal_growth=0.04
```

CAPM & WACC

```
cost_equity=risk_free+beta*(market_return-risk_free)
wacc=equity_weight*cost_equity+debt_weight*cost_debt*(1-tax_rate)
print('Cost of Equity',round(cost_equity,4))
print('WACC',round(wacc,4))
```

```
Cost of Equity 0.13
WACC 0.116
```

Discount Cash Flows

```
fcf=forecast['FCF'].values
pv=[]
for i,x in enumerate(fcf,1):
    pv.append(x/((1+wacc)**i))
forecast['PV_FCF']=pv

forecast_display = forecast.copy()
forecast_display[['FCF','PV_FCF']] = forecast_display[['FCF','PV_FCF']] / 1e7
display(forecast_display.style.format({'FCF': '₹{:,.2f} Cr', 'PV_FCF': '₹{:,.2f} Cr'}))
```

	Forecast Revenue	Operating Income	CapEx	Depreciation	Tax	
Year 1	3181272634865.780762	1018007243157.050049	127250905394.631256	47719089522.986717	254501810789.262512	₹68
Year 2	4267535561271.039062	1365611379606.732422	170701422450.841553	64013033419.065582	341402844901.683105	₹91
Year 3	5724708900179.278320	1831906848057.368896	228988356007.171173	85870633502.689178	457976712014.342285	₹123
Year 4	7679442038915.540039	2457421452452.972656	307177681556.621582	115191630583.733093	614355363113.243164	₹165
Year 5	10301629490228.998047	3296521436873.279785	412065179609.159912	154524442353.434967	824130359218.319824	₹221

Terminal Value

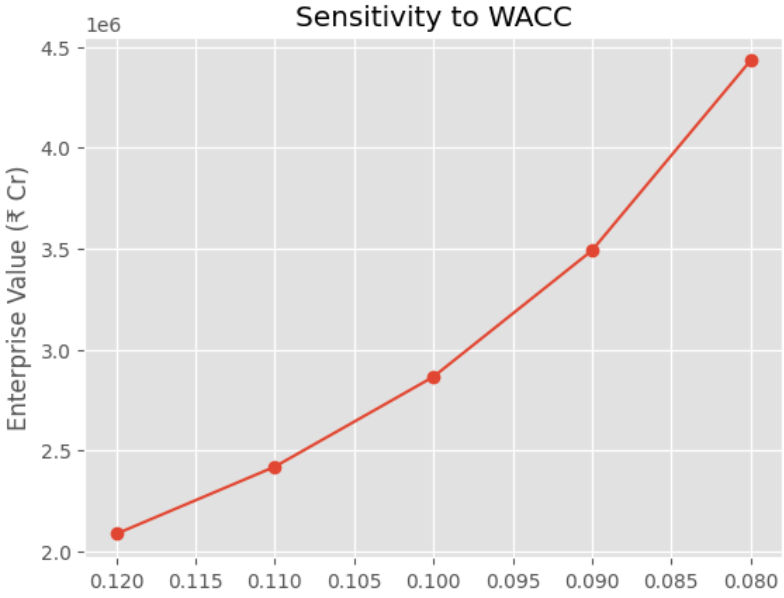
```
tv=fcf[-1]*(1+terminal_growth)/(wacc-terminal_growth)
pv_tv=tv/((1+wacc)**len(fcf))
ev=sum(pv)+pv_tv
net_debt=0
equity=ev-net_debt
print(f"Enterprise Value: ₹{ev/1e7:,.2f} Cr")
print(f"Equity Value: ₹{equity/1e7:,.2f} Cr")
```

```
Enterprise Value: ₹2,208,722.87 Cr
Equity Value: ₹2,208,722.87 Cr
```

Sensitivity

```
rates=np.arange(0.08,0.13,0.01)
vals=[]
for r in rates:
    tv=fcf[-1]*(1+terminal_growth)/(r-terminal_growth)
    vals.append(sum([fcf[i]/((1+r)**(i+1)) for i in range(len(fcf))])+tv/((1+r)**len(fcf)))
sens=pd.DataFrame({'WACC':rates,'EnterpriseValue':vals})
sens_display = sens.copy()
sens_display['EnterpriseValue'] = sens_display['EnterpriseValue'] / 1e7

sens_display = sens_display.style.format({
    'EnterpriseValue': '₹{:,.2f} Cr',
    'WACC': '{:.0%}'
})
display(sens_display)

plt.plot(sens['WACC'], sens['EnterpriseValue']/1e7, marker='o')
plt.ylabel("Enterprise Value (₹ Cr)")
plt.gca().invert_xaxis()
plt.title('Sensitivity to WACC')
plt.show()
```

	WACC	EnterpriseValue
0	8%	₹4,431,011.42 Cr
1	9%	₹3,490,095.32 Cr
2	10%	₹2,864,538.92 Cr
3	11%	₹2,419,117.11 Cr
4	12%	₹2,086,220.93 Cr



Recommendation

```
print(f"Intrinsic Enterprise Value: ₹{ev/1e7:,.2f} Cr")
print('Further enhancement: compare intrinsic value with live market capitalization to produce BUY/HOLD/SI
```

```
Intrinsic Enterprise Value: ₹2,208,722.87 Cr
Further enhancement: compare intrinsic value with live market capitalization to produce BUY/HOLD/SELL recor
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
plt.style.use('ggplot')
```

Load DCF Forecast

```
forecast = pd.read_csv("Forecast_Model.csv", index_col=0)
fcf = forecast["FCF"].values
```

Monte Carlo DCF Simulation

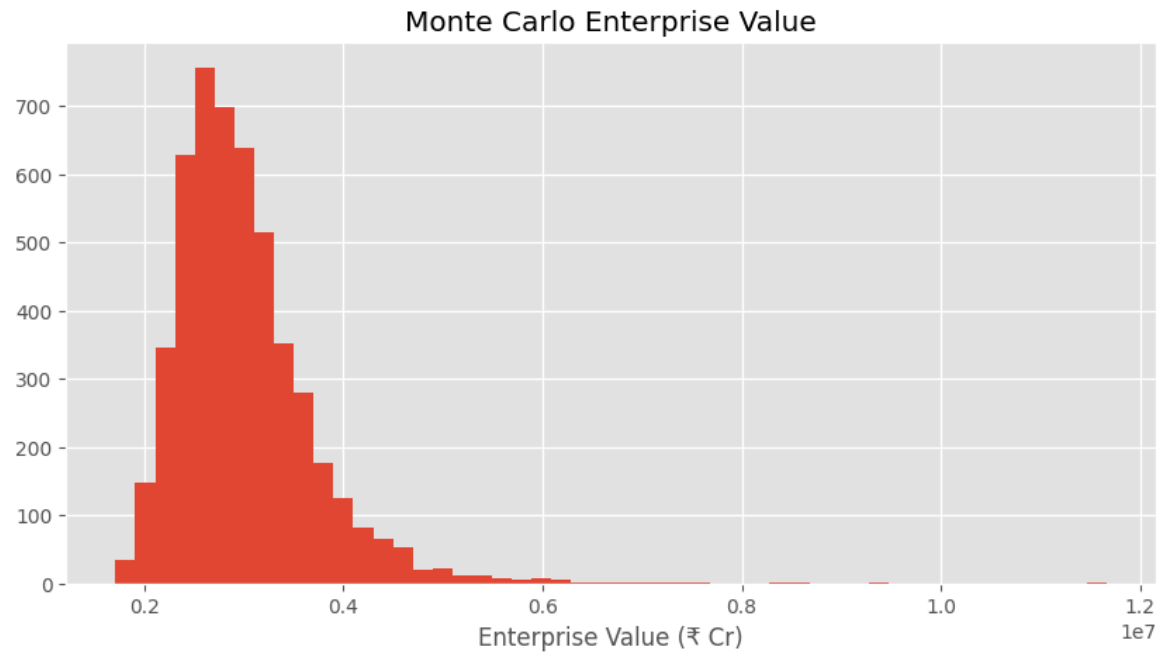
```
np.random.seed(42)

iterations = 5000
values = []

for _ in range(iterations):
    wacc = np.random.normal(0.10,0.01)
    g = np.random.normal(0.04,0.005)

    pv = sum([fcf[i]/((1+wacc)**(i+1)) for i in range(len(fcf))])

    tv = fcf[-1]*(1+g)/(wacc-g)
```

```
        ev = pv + tv/((1+wacc)**len(fcf))

        values.append(ev)

    values=np.array(values)

    print(f"Mean EV: ₹{values.mean()/1e7:,.2f} Cr")
    print(f"Median EV: ₹{np.median(values)/1e7:,.2f} Cr")
```

```
Mean EV: ₹2,989,109.06 Cr
Median EV: ₹2,869,919.84 Cr
```

Distribution

```
plt.figure(figsize=(10,5))
plt.hist(values/1e7, bins=50)
plt.xlabel('Enterprise Value (₹ Cr)')
plt.title("Monte Carlo Enterprise Value")
plt.show()
```



Bull / Base / Bear

```
scenarios=pd.DataFrame({
'Scenario':['Bear','Base','Bull'],
'Revenue Growth':[0.08,0.12,0.16],
'Operating Margin':[0.28,0.32,0.36],
'WACC':[0.11,0.10,0.09]
})

display(scenarios)
```

	Scenario	Revenue Growth	Operating Margin	WACC
0	Bear	0.08	0.28	0.11
1	Base	0.12	0.32	0.10
2	Bull	0.16	0.36	0.09

Next steps: Generate code with `scenarios` New interactive sheet

Sensitivity Matrix

```
waccs=np.arange(0.08,0.13,0.01)
growth=np.arange(0.03,0.06,0.01)

matrix=pd.DataFrame(index=[round(i,2) for i in growth],
columns=[round(i,2) for i in waccs])

for g in growth:
    for w in waccs:
        tv=fcf[-1]*(1+g)/(w-g)
        matrix.loc[round(g,2), round(w,2)] = round(tv/1e7, 2)

display(matrix.style.format("₹{:,.2f} Cr"))
```

	0.080000	0.090000	0.100000	0.110000	0.120000
0.030000	₹4,562,591.70 Cr	₹3,802,159.75 Cr	₹3,258,994.07 Cr	₹2,851,619.81 Cr	₹2,534,773.17 Cr
0.040000	₹5,758,610.89 Cr	₹4,606,888.71 Cr	₹3,839,073.92 Cr	₹3,290,634.79 Cr	₹2,879,305.44 Cr
0.050000	₹7,751,976.19 Cr	₹5,813,982.14 Cr	₹4,651,185.71 Cr	₹3,875,988.10 Cr	₹3,322,275.51 Cr

Peer Valuation

```
peer=pd.DataFrame({
'Company':['HDFC Bank','ICICI Bank','Axis Bank','Kotak Bank','SBI'],
'PE':[20,22,16,24,10],
'PB':[3.0,3.5,2.2,3.7,1.6],
'ROE':[16,18,15,14,17]
})

display(peer)

peer.plot(x='Company',kind='bar',figsize=(10,5))
plt.show()
```

	Company	PE	PB	ROE
0	HDFC Bank	20	3.0	16
1	ICICI Bank	22	3.5	18
2	Axis Bank	16	2.2	15
3	Kotak Bank	24	3.7	14
4	SBI	10	1.6	17